Visión por Computadora II - CEAI - FIUBA

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

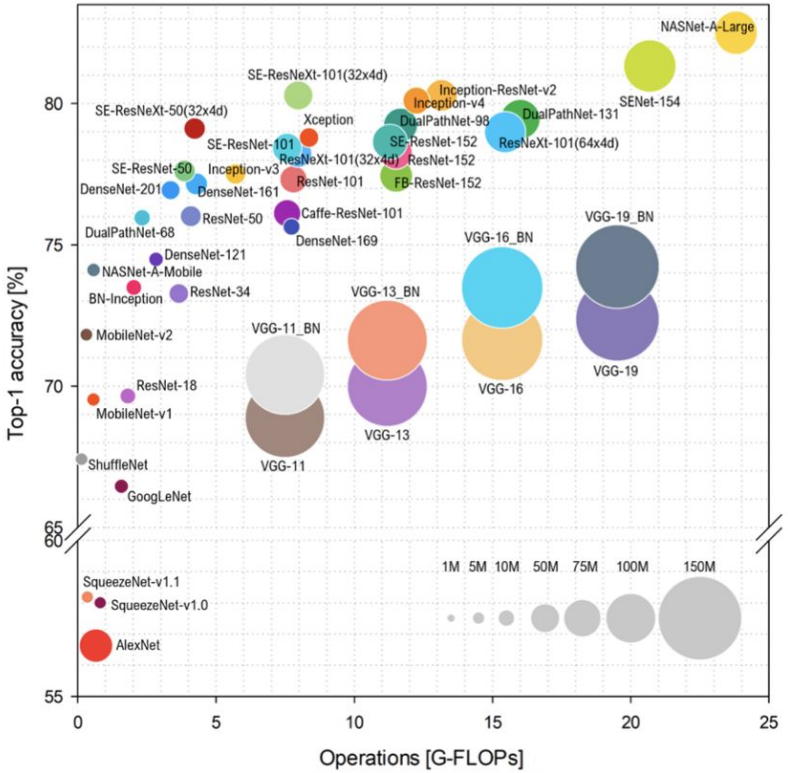
Profesores:

- Cavalieri Juan Ignacio - juanignaciocavalieri@gmail.com
- Cornet Juan Ignacio - juanignaciocornet@gmail.com
- Khodadad Pakdaman - khodadad.pakdaman@gmail.com

Sexta clase:

- Aprendizaje autosupervisado
  - SimCLR
  - SwAV
- Vision Transformers
- Optimización de hiperparametros
- AutoAugment

Pre-entrenamiento.

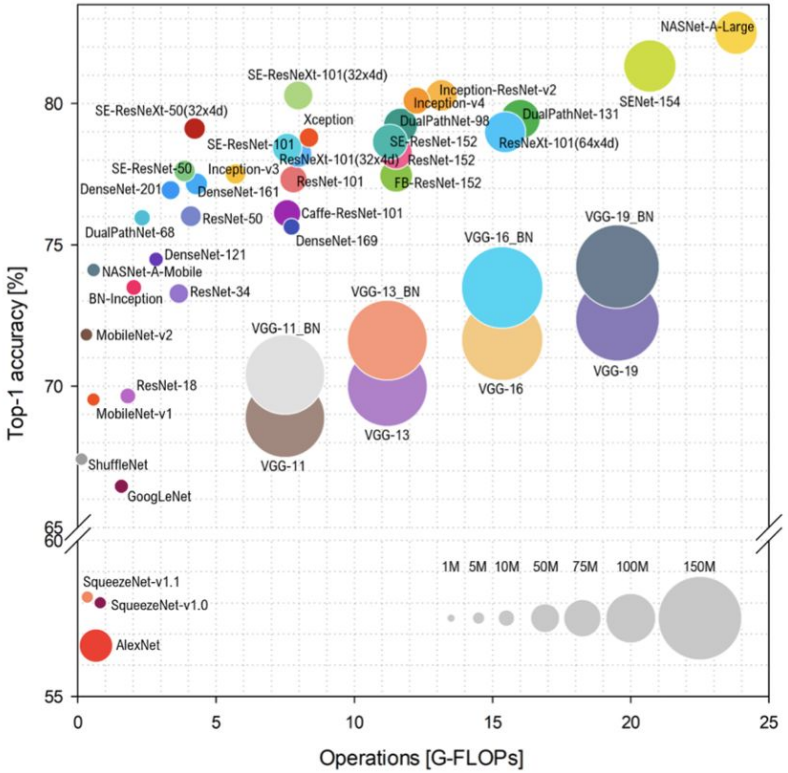


Con el desarrollo del aprendizaje profundo y gracias a los grandes avances en el poder de cómputo de las procesadores actuales, los modelos que se utilizan en visión por computadora han crecido enormemente en cantidad de parámetros.

Esta gran cantidad de parámetros, le dan gran expresividad a las redes, pero dificulta el aprendizaje, ya que requiere que se utilicen técnicas que eviten el sobreentrenamiento, y sobre todo MUCHOS datos.

La práctica habitual es utilizar redes pre-entrenadas y aplicar transferencia de aprendizaje.

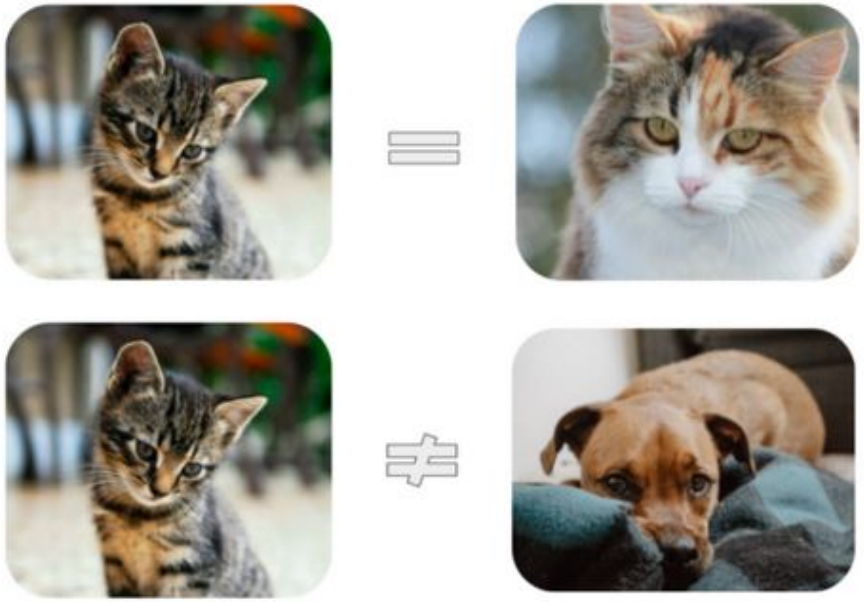
Pre-entrenamiento.



Sin embargo, las técnicas de pre-entrenamiento supervisado, se encuentran con su límite: conseguir suficiente número de imágenes etiquetadas para poder realizar el entrenamiento de modelos ÁVIDOS de datos.

Los métodos de entrenamiento semi supervisados permiten aprovechar grandes volúmenes de imágenes para entrenamiento, sin requerir que los mismos se encuentren anotados. De esta manera, los modelos pueden internalizar patrones y representaciones comunes a un gran conjunto de imágenes, y luego hacer transferencia de estilo utilizando un pequeño volumen de imágenes etiquetadas.

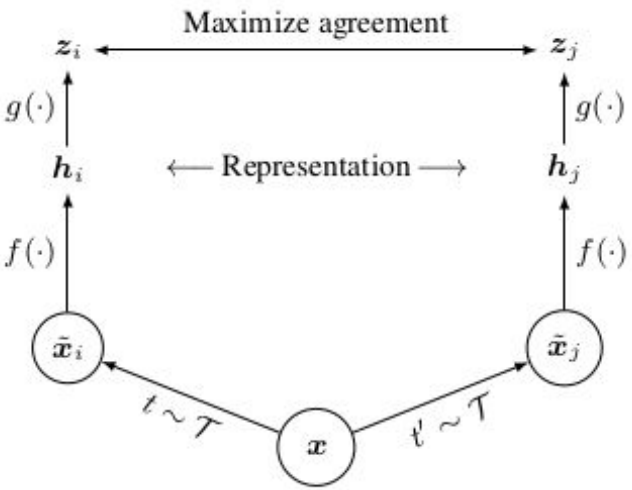
Aprendizaje contrastivo - SimCLR



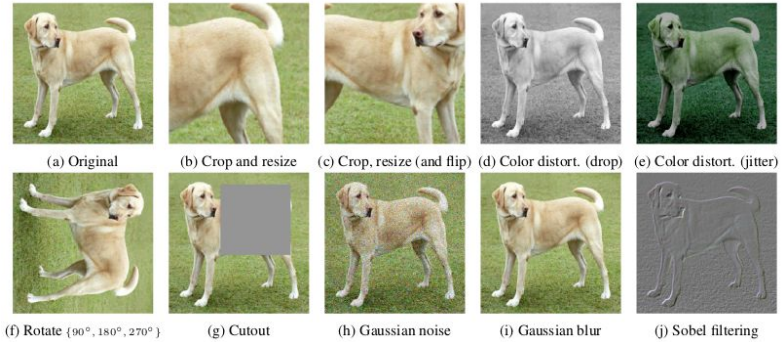
El aprendizaje contrastivo es una técnica que permite el aprendizaje de features de un modelo sin necesidad de etiquetado.
El no uso de etiquetas es lo que lo hace tan poderoso, ya que generalmente es el cuello de botella en la implementación de muchos proyectos de ML.
El objetivo es encontrar un modelo que produzca features similares cuando las entradas son de la misma clase y más difíciles cuando las clases son diferentes.

A Simple Framework for Contrastive Learning of Visual Representations Link

Aprendizaje contrastivo - SimCLR

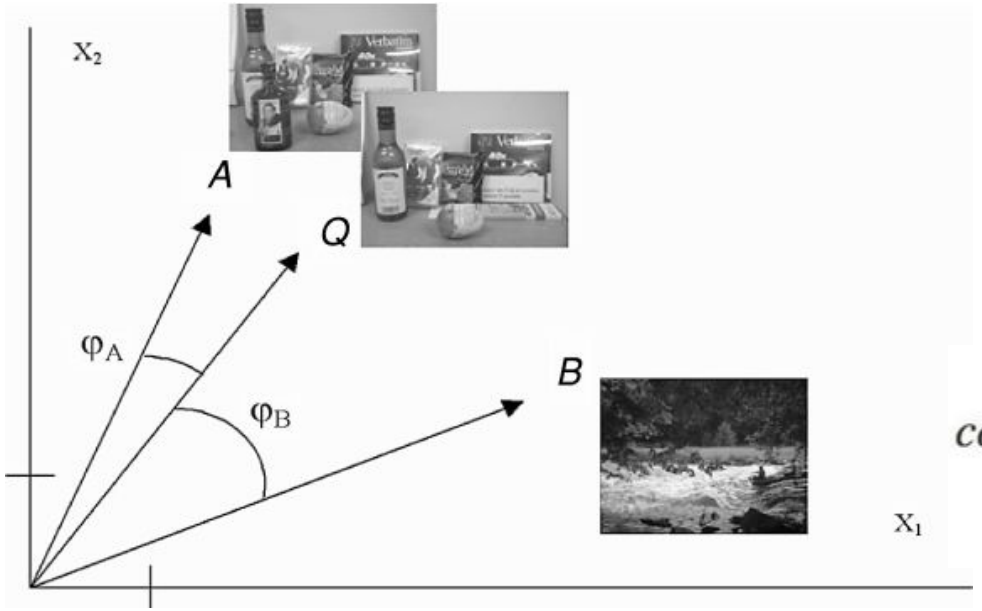


Para ello se aplican técnicas de data augmentation sobre cada una de las imágenes, y se busca que las imágenes que provienen de una misma fuente tengan salidas similares, mientras que imágenes que provengan de distinta fuente tengan salidas más alejadas entre sí en el espacio de features.



A Simple Framework for Contrastive Learning of Visual Representations [Link]

Aprendizaje contrastivo - SimCLR

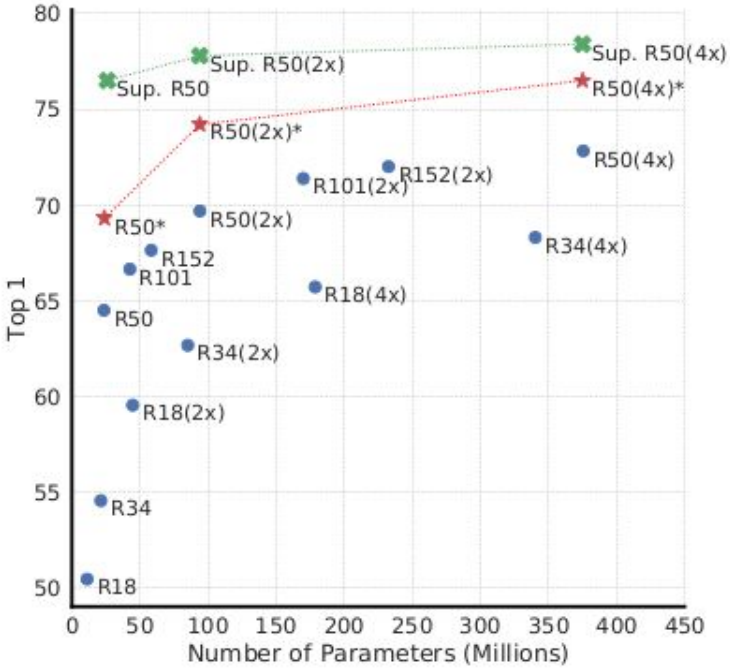


Para cuantificar la distancia entre las imágenes se usa similitud de cosenos, y luego se usa una función de costo para minimizar que en el caso de SimCLR es NT-Xent

$$cos(\theta) = \frac{A \cdot B}{\|A \| B\|} = \frac{\sum_i A_i B_i}{\sqrt{\sum_i A_i^2}\sqrt{\sum_i B_i^2}}$$

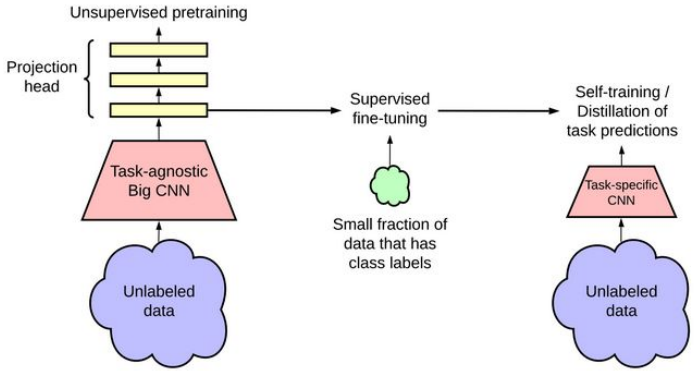
A Simple Framework for Contrastive Learning of Visual Representations [Link]

Aprendizaje contrastivo - SimCLR



A Simple Framework for Contrastive Learning of Visual Representations [Link]

Aprendizaje contrastivo - SimCLR



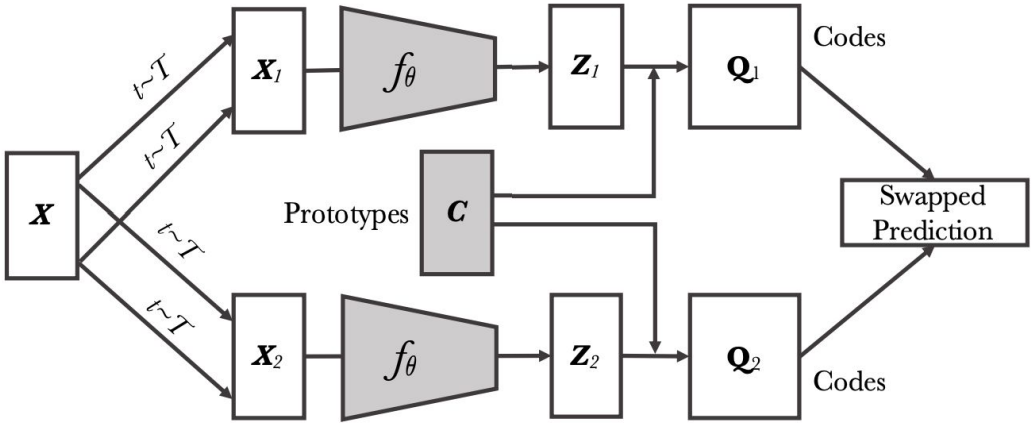
Una extensión de su uso es el aprendizaje semi supervisado, en el cual se usa una red pre-entrenada de con el framework de contrastive learning y luego se le aplica un fine tuning supervisado, pero con una cantidad de datos etiquetados mucho menor al que se necesitaría para entrenar la red completa.

A Simple Framework for Contrastive Learning of Visual Representations

	Food	CIFAR10	CIFAR100	Birdsnap	SUN397	Cars	Aircraft	VOC2007	DTD	Pets	Caltech-101	Flowers
Linear evaluation:												
SimCLR (ours)	**76.9**	**95.3**	80.2	48.4	**65.9**	60.0	61.2	**84.2**	**78.9**	89.2	**93.9**	**95.0**
Supervised	75.2	**95.7**	**81.2**	**56.4**	64.9	**68.8**	**63.8**	83.8	**78.7**	**92.3**	**94.1**	94.2
Fine-tuned:												
SimCLR (ours)	**89.4**	**98.6**	**89.0**	**78.2**	**68.1**	**92.1**	**87.0**	**86.6**	**77.8**	92.1	**94.1**	97.6
Supervised	88.7	98.3	**88.7**	**77.8**	67.0	91.4	**88.0**	86.5	**78.8**	**93.2**	**94.2**	**98.0**
Random init	88.3	96.0	81.9	**77.0**	53.7	91.3	84.8	69.4	64.1	82.7	72.5	92.5

A Simple Framework for Contrastive Learning of Visual Representations [Link]

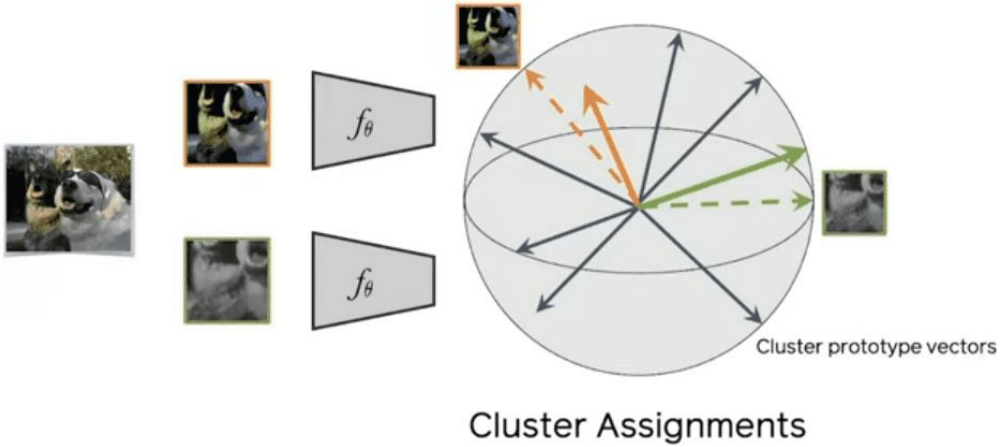
Aprendizaje contrastivo - SWAV



Se propone un algoritmo online, SwAV, que aprovecha las ventajas de los métodos contrastivos sin necesidad de calcular comparaciones por pares. En concreto, dicho método agrupa simultáneamente los datos al tiempo que refuerza la coherencia entre las asignaciones de grupos producidas para diferentes vistas de la misma imagen, en lugar de comparar las características directamente como en el aprendizaje contrastivo.

Unsupervised Learning of Visual Features by Contrasting Cluster Assignments Link

Aprendizaje contrastivo - SWAV



Intuición: Si z_t y z_s capturan información similar, podemos predecir el código q_s a partir de la otra característica z_t. En otras palabras, si las dos vistas comparten la misma semántica, sus objetivos serán similares. Esta es la idea del "intercambio".

- Crear vistas N a partir de la imagen de entrada X.
- Calcular las representaciones de las características de la imagen z
- Calcular las similitudes normalizadas por softmax entre todos los z y c.
- Calcular la matriz de códigos Q de forma iterativa.
- Calcular la *cross entropy loss* entre la representación t, y el código de la representación s.
- Promediar las funciones de pérdida entre todas las vistas N.

Unsupervised Learning of Visual Features by Contrasting Cluster Assignments Link

Aprendizaje contrastivo - SWAV

Q es la matriz que contiene a los vectores de código que servirán como objetivo para el entrenamiento $\mathbf{Q} = [\mathbf{q}_1, \ldots, \mathbf{q}_B]$

La misma se puede definir como el transporte óptimo entre las features Z y los prototipos C, dado como costo el de la matriz de similitud.

$$\max_{\mathbf{Q} \in \mathcal{Q}} \operatorname{Tr}\left(\mathbf{Q}^\top \mathbf{C}^\top \mathbf{Z}\right) + \varepsilon H(\mathbf{Q}),$$

$$\mathcal{Q} = \left\{ \mathbf{Q} \in \mathbb{R}_+^{K \times B} \mid \mathbf{Q}\mathbf{1}_B = \frac{1}{K}\mathbf{1}_K, \mathbf{Q}^\top \mathbf{1}_K = \frac{1}{B}\mathbf{1}_B \right\},$$

A la matriz Q se le imponen restricciones de equivariancia para evitar colapsos en un único prototipo.

La resolución del problema de optimización se realiza mediante la aproximación de **Sinkhorn-Knopp**

$$\mathbf{Q}^* = \operatorname{Diag}(\mathbf{u}) \exp\left(\frac{\mathbf{C}^\top \mathbf{Z}}{\varepsilon}\right) \operatorname{Diag}(\mathbf{v}),$$

Una vez obtenidos los codigos Q, la funcion de perdida alternada es la siguiente

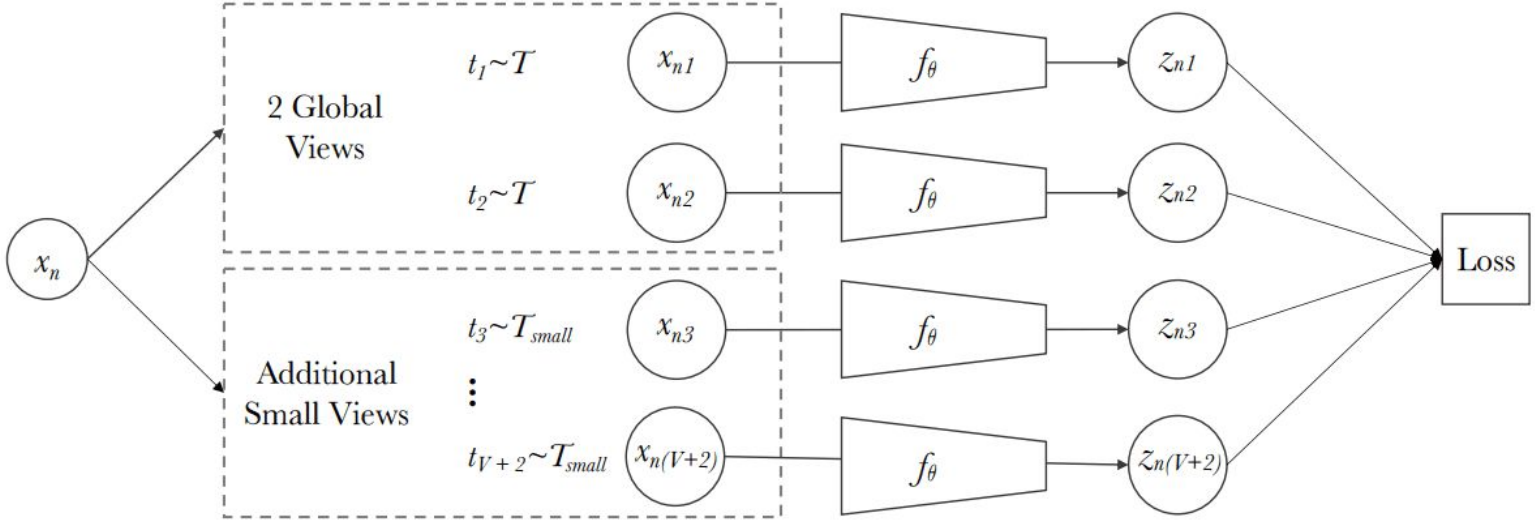
$$L(\mathbf{z}_t, \mathbf{z}_s) = \ell(\mathbf{z}_t, \mathbf{q}_s) + \ell(\mathbf{z}_s, \mathbf{q}_t),$$

$$\ell(\mathbf{z}_t, \mathbf{q}_s) = -\sum_k \mathbf{q}_s^{(k)} \log \mathbf{p}_t^{(k)}, \quad \text{where} \quad \mathbf{p}_t^{(k)} = \frac{\exp\left(\frac{1}{\tau}\mathbf{z}_t^\top \mathbf{c}_k\right)}{\sum_{k'} \exp\left(\frac{1}{\tau}\mathbf{z}_t^\top \mathbf{c}_{k'}\right)}.$$

Unsupervised Learning of Visual Features by Contrasting Cluster Assignments Link

Aprendizaje contrastivo - SWAV

$$L(\mathbf{z}_{t_1}, \mathbf{z}_{t_2}, \ldots, \mathbf{z}_{t_{V+2}}) = \sum_{i \in \{1,2\}} \sum_{v=1}^{V+2} \mathbf{1}_{v \neq i} \ell(\mathbf{z}_{t_v}, \mathbf{q}_{t_i}).$$



Unsupervised Learning of Visual Features by Contrasting Cluster Assignments Link

Method	Frozen	Finetuned
Random	15.0	76.5
MoCo	-	77.3*
SimCLR	60.4	77.2
SwAV	**66.5**	**77.8**

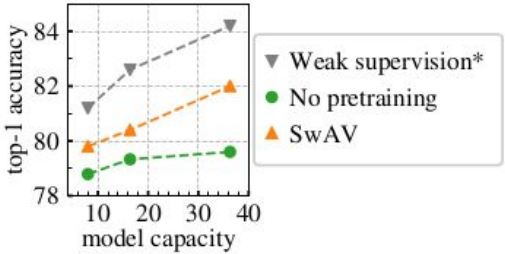
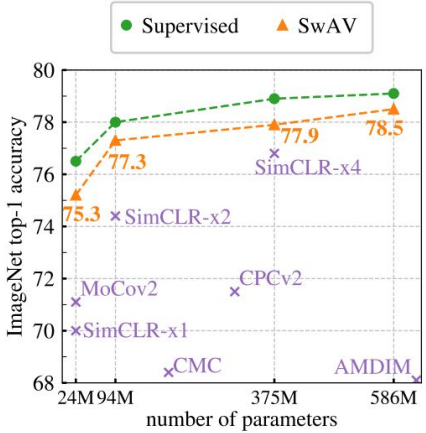


Figure 4: **Pretraining on uncurated data.** Top-1 accuracy on ImageNet for pretrained models on an uncurated set of 1B random Instagram images. **(left)** We compare ResNet-50 pretrained with either SimCLR or SwAV on two downstream tasks: linear classification on frozen features or finetuned features. **(right)** Performance of finetuned models as we increase the capacity of a ResNext following [41]. The capacity is provided in billions of Mult-Add operations.
*: *pretrained on a curated set of 1B Instagram images filtered with 1.5k hashtags similar to ImageNet classes.*

	Arch.	Param.	Top1
Supervised	R50	24	76.5
Colorization [65]	R50	24	39.6
Jigsaw [46]	R50	24	45.7
NPID [58]	R50	24	54.0
BigBiGAN [15]	R50	24	56.6
LA [68]	R50	24	58.8
NPID++ [44]	R50	24	59.0
MoCo [24]	R50	24	60.6
SeLa [2]	R50	24	61.5
PIRL [44]	R50	24	63.6
CPC v2 [28]	R50	24	63.8
PCL [37]	R50	24	65.9
SimCLR [10]	R50	24	70.0
MoCov2 [11]	R50	24	71.1
SwAV	R50	24	**75.3**



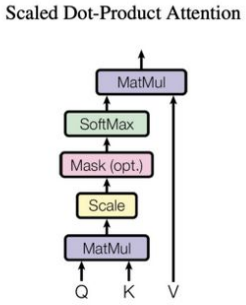
Unsupervised Learning of Visual Features by Contrasting Cluster Assignments Link

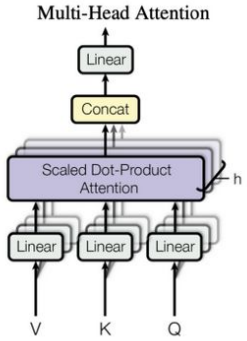
Table 1: **Semi-supervised learning on ImageNet with a ResNet-50.** We finetune the model with 1% and 10% labels and report top-1 and top-5 accuracies. *: *uses RandAugment [12].*

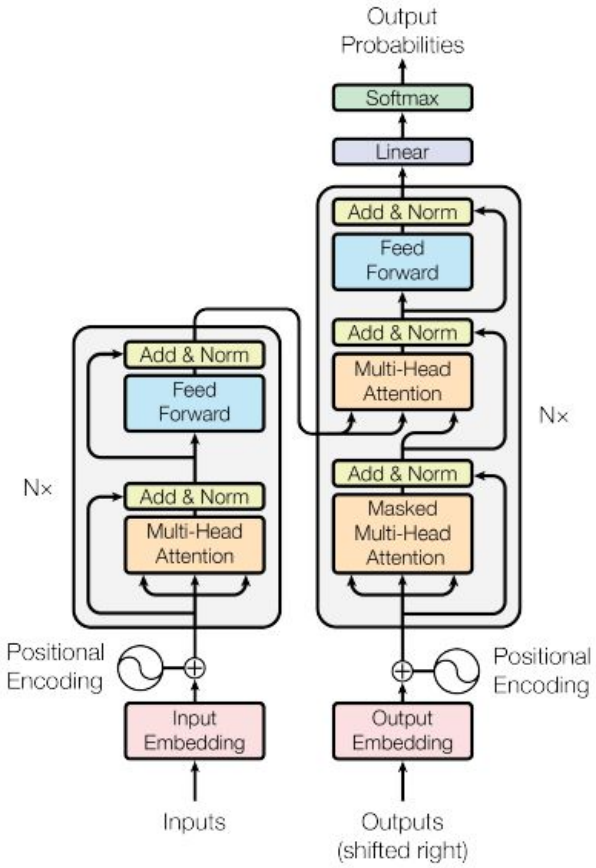
	Method	1% labels Top-1	1% labels Top-5	10% labels Top-1	10% labels Top-5
	Supervised	25.4	48.4	56.4	80.4
Methods using label-propagation	UDA [60]	-	-	68.8*	88.5*
	FixMatch [51]	-	-	**71.5***	89.1*
Methods using self-supervision only	PIRL [44]	30.7	57.2	60.4	83.8
	PCL [37]	-	75.6	-	86.2
	SimCLR [10]	48.3	75.5	65.6	87.8
	SwAV	**53.9**	**78.5**	70.2	**89.9**

Unsupervised Learning of Visual Features by Contrasting Cluster Assignments Link

Vision Transformers







El transformer es un modelo que basa sus predicciones en el mecanismo de auto atención, cuyas operaciones fundamentales son sumas ponderadas y activaciones

Attention is all you need [Link]

Vision Transformers

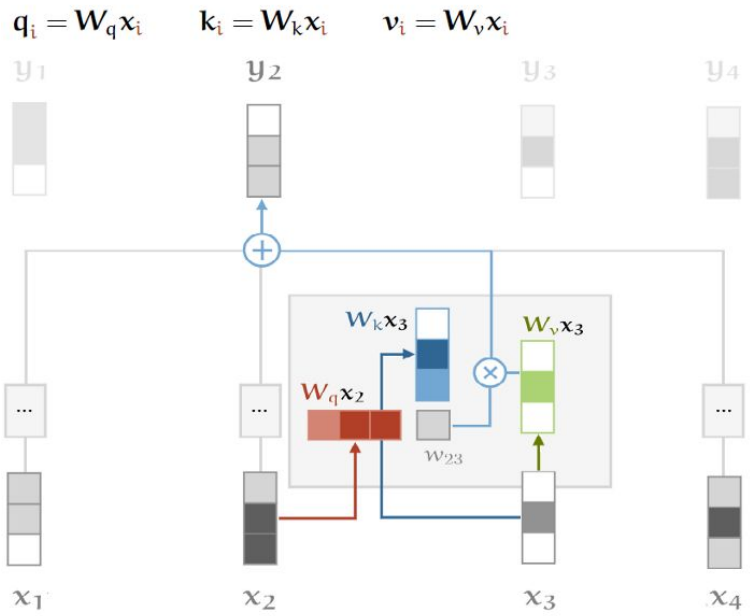


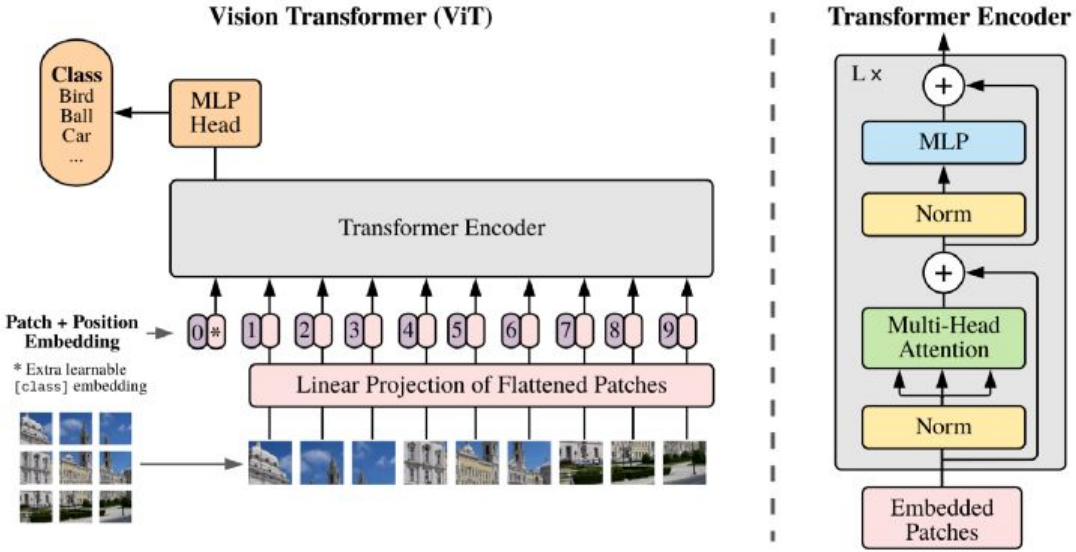
Illustration of the self-attention with key, query and value

Attention is all you need [Link]

Inicialmente desarrollado para NLP, tratando el problema de traducción secuencia a secuencia.

El mecanismo de atención analiza la relevancia de cada uno de los elementos de la secuencia para la traducción de un elemento dado.

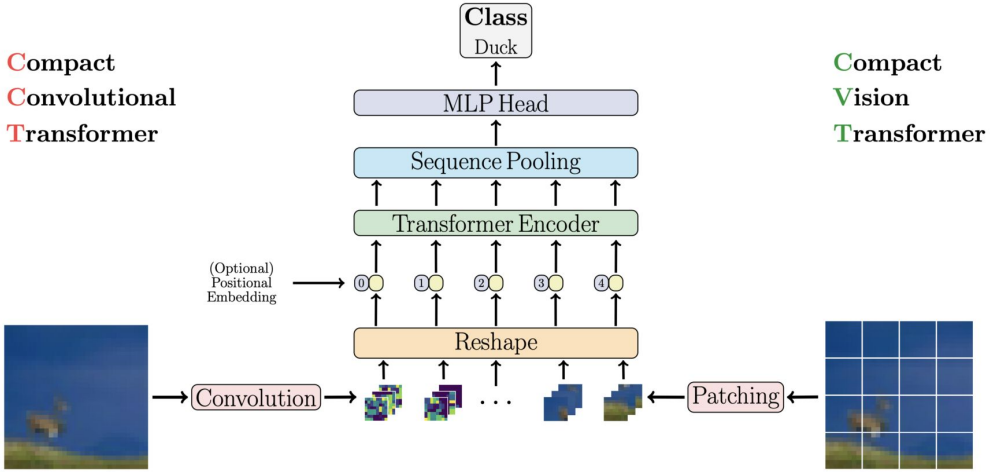
Vision Transformers



	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	**88.55** ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	**90.72** ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	**99.50** ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	–
CIFAR-100	**94.55** ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	–
Oxford-IIIT Pets	**97.56** ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	–
Oxford Flowers-102	99.68 ± 0.02	**99.74** ± 0.00	99.61 ± 0.02	99.63 ± 0.03	–
VTAB (19 tasks)	**77.63** ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	–
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

An image is worth 16 x 16 words [Link]

Vision Transformers



Model	Top-1	# Params	MACs	Training Epochs
ResNet152 [7]	78.57%	60.19 M	11.58 G	≈ 120
ViT-Base/16 [2] ↑ 384	77.91%	86.83 M	49.35 G	300
CCT-14/7×2⋆	80.67%	**22.36** M	**5.11** G	300♡
CCT-14/7×2⋆ ↑ 384	**82.71%**	22.51 M	15.02 G	300♡

Escaping the Big Data Paradigm with Compact Transformers Link

Vision Transformers

Transformers explained:
https://towardsdatascience.com/transformers-explained-visually-not-just-how-but-why-they-work-so-well-d840bd61a9d3
https://towardsdatascience.com/vision-transformers-in-pytorch-43d13cb7ec7a
https://medium.com/pytorch/training-compact-transformers-from-scratch-in-30-minutes-with-pytorch-ff5c21668ed5

Ajuste de hiperparametros

¿Cómo elegir los **mejores** hiperparametros para nuestro problema?

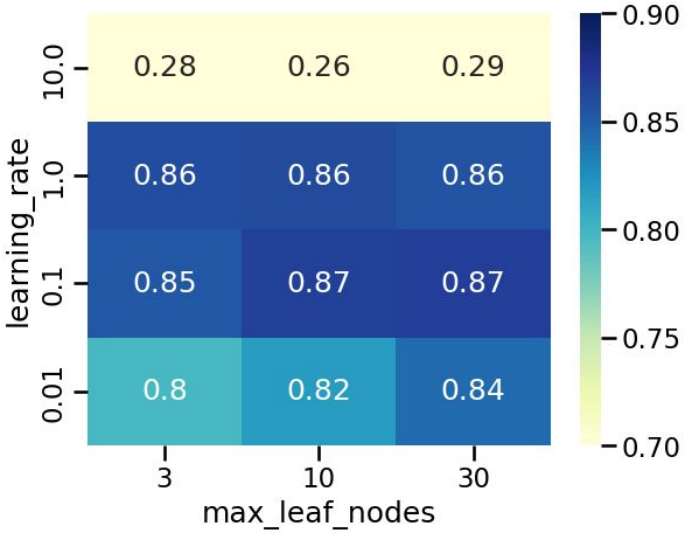
- layers
- kernels
- learning_rate
- data_augmentation
- regularization

Cual es la métrica adecuada para definir el buen desempeño de una solución a nuestro problema?
Definir métrica a optimizar.

Solución impulsiva. Probar a mano distintos valores de hiperparametros.
Cansador, ineficiente,

Grid Search

Solución más natural: hacer una búsqueda exhaustiva. Es decir probando con todos los valores de los hiperparámetros que podamos y eligiendo la mejor combinación. Este método se llama **Grid Search** (“búsqueda de cuadrícula”).



Grid Search

Entonces, una vez que sabemos qué métrica queremos optimizar,
Grid Search consiste en:

1. Elegimos los valores que puede tomar cada hiperparámetro
2. Armamos las combinaciones “todos con todos” → **Tenemos nuestra grilla.**
3. Recorremos la grilla, entrenamos un modelo para cada combinación y lo evaluamos.
4. Elegimos los hiperparámetros que definen el mejor modelo.

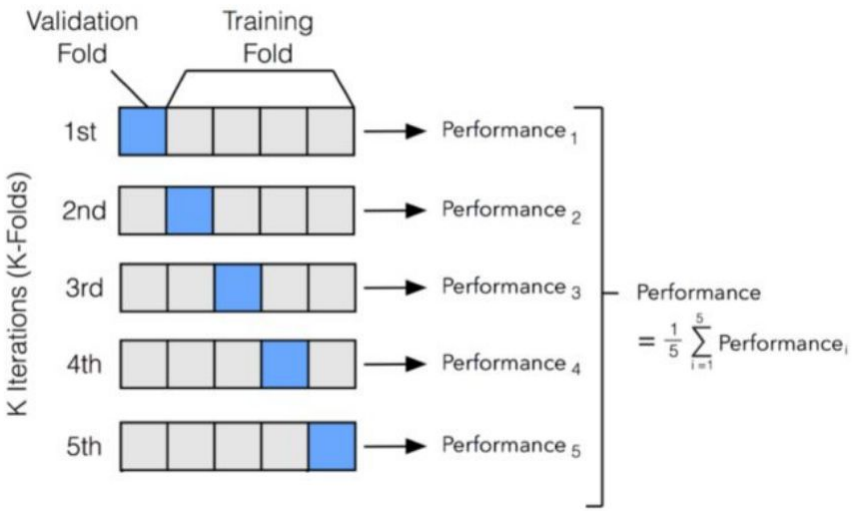
Recomendación: hacer una primera búsqueda en grilla, usando valores para los parámetros que sigan una progresión geométrica. Luego hacer una segunda búsqueda, más localizada en las cercanías de los parámetros con lo que mejor performance se obtuvo.

Grid Search + CV

¿Como hacemos para evaluar los modelos?
¿Con Train/Test split?
¿Con Validación Cruzada?

Al estar probando MUCHOS modelos, podría suceder que uno se desempeñe muy bien en el conjunto de Train simplemente por azar.

Por esto ,es muy importante evaluar cada modelo creado por Grid Search con Validación Cruzada en el conjunto de entrenamiento.



Grid Search + CV

1. Entrenamos un modelo con los hiperparámetros que dieron el mejor resultado en validacion con todo el conjunto de entrenamiento.
2. Evaluamos su performance en el conjunto de testeo

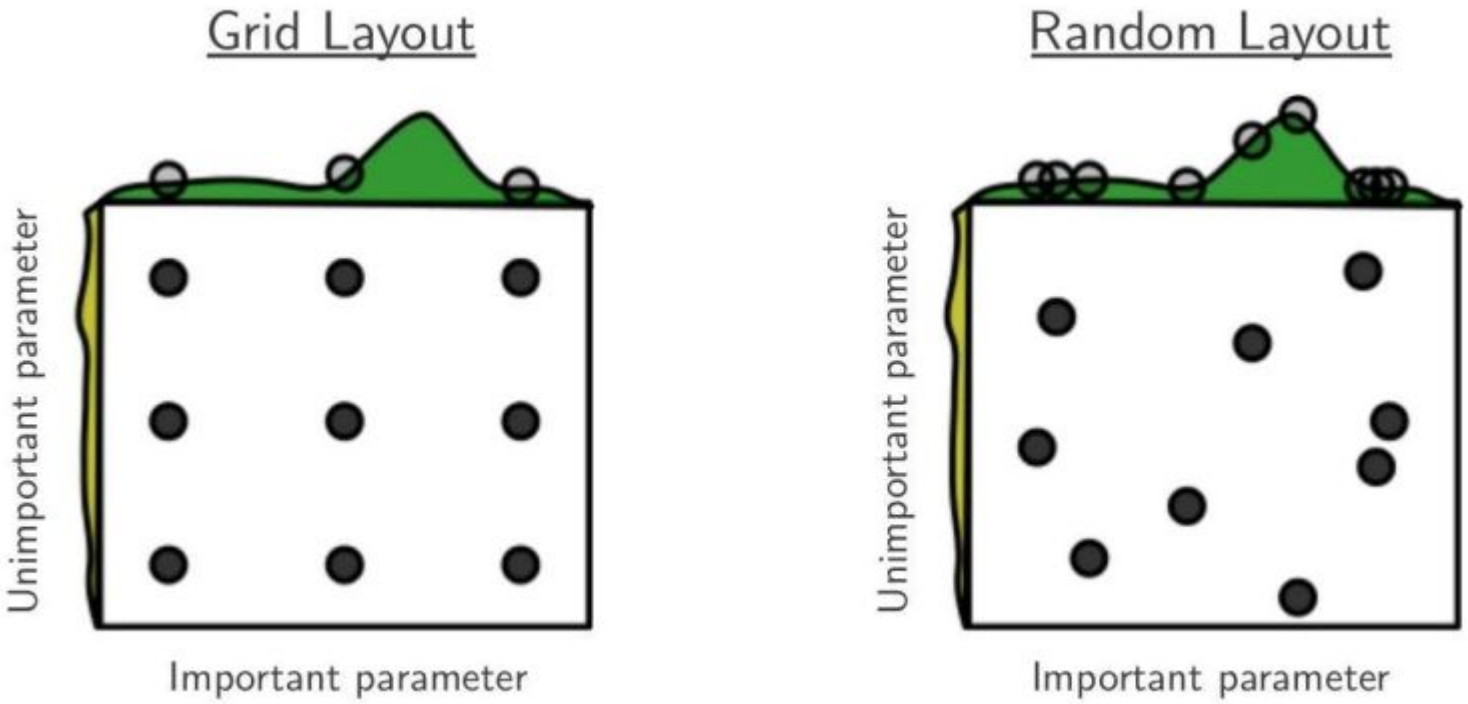
Como la evaluación de performance de CV puede estar sesgada, es importante evaluar al final en este conjunto para obtener una estimación menos sesgada del desempeño.

Grid Search + CV puede ser computacionalmente muy demandante.
Y tal vez al “probar” con pocos valores de los hiperparámetros nos estamos perdiendo algunas cosas importantes.
Además, suele haber hiperparámetros mucho más importantes que otros.

Random Search

En este caso se exploran opciones y combinaciones de hiperparametros al azar, de manera menos “ordenada”. En muchas circunstancias, ¡esto es más eficiente, tanto desde el punto de vista de performance del modelo como de desempeño computacional!

Al estar recorriendo el espacio de hiperparametros de manera aleatoria, se logran testear muchos más valores, y debido a que generalemente hay hiperparametros que son más importantes que otros para la performance, se lograría una mejor eficiencia en el entrenamiento.


Grid Layout
Unimportant parameter
Important parameter
Random Layout
Unimportant parameter
Important parameter

Random + Grid Search

Random Search o Grid Search?

Para qué elegir?

Una práctica habitual consiste en realizar una búsqueda aleatoria de hiperparametros, y luego en base a los conjuntos de hiperparametros que mejor hayan preformado, realizar una búsqueda exhaustiva usando GridSearch.

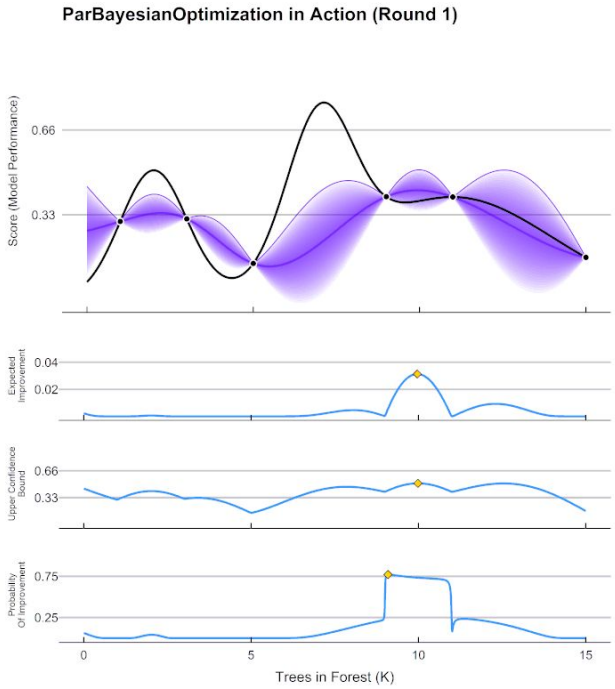
De esta manera, se muestrea mayor cantidad de valores para los hiperparametros más importantes, y por otro lado se hace una búsqueda en detalle más localizada, logrando ahorros en costo computacional y sobretodo en tiempo.

Optimización de Hiperparametros Automática

A diferencia de los casos anteriores, en la búsqueda automática de hiperparametros, los mismos son identificados mediante el uso de técnicas de optimización como son:

- Optimizacion Bayesiana
- Descenso en gradiente
- Algoritmos geneticos

Bayesian Optimization



En el caso de la optimización bayesiana, se usa una enfoque probabilístico para encontrar el mínimo de una función. Entonces para conformar una búsqueda de hiperparametros son estos algoritmos es necesario determinar:

- Función objetivo.
- Espacio de hiperparametros.
- Algoritmo de optimización.

La optimización bayesiana puede reducir el número de iteraciones de búsqueda eligiendo los valores de entrada teniendo en cuenta los resultados anteriores. De este modo, podemos concentrar nuestra búsqueda desde el principio en los valores que se acercan más a nuestro resultado deseado.

By AnotherSamWilson - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=84842869

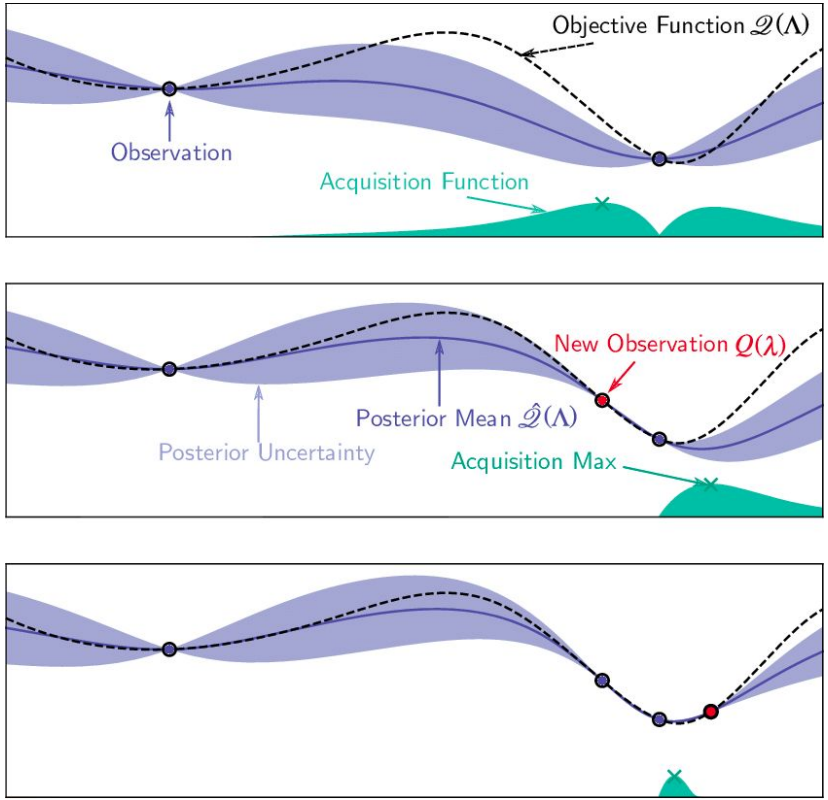
Bayesian Optimization

Implementado en bibliotecas como scikit-optimize, ray[tune], hyperopt…

1. Seleccionar una muestra de parámetros, con la función de adquisición.
2. Evaluar la muestra de parámetros con la función objetivo.
3. Actualizar los datos y así también la aproximación a priori
4. Volver a 1.

Para la aproximación de la función objetivo se suele usar un Proceso Gaussiano.

Como función de adquisición hay técnicas que permiten minimizar la creencia a priori para seleccionar los parámetros de interés.



https://machinelearningmastery.com/what-is-bayesian-optimization/

Herramientas disponibles

Skorch https://github.com/skorch-dev
Optuna https://optuna.org/
Tune https://docs.ray.io/

search.py

```python
import torch
import torch.nn as nn

from skorch import NeuralNetClassifier
from sklearn.model_selection import GridSearchCV

from utils import creat_search_run, save_best_hyperparam
from model import CustomNet
from datasets import get_data_loaders
```

```python
# Instance of `NeuralNetClassifier` to be passed to `GridSearchCV`
net = NeuralNetClassifier(
    module=CustomNet, max_epochs=epochs,
    optimizer=torch.optim.Adam,
    criterion=criterion,
    lr=lr, verbose=1
)

# Get the training and validation data loaders.
train_loader, valid_loader, dataset_classes = get_data_loaders(224)


params = {
'lr': [0.001, 0.01, 0.005, 0.0005],
'max_epochs': list(range(20, 55, 5)),
'module__first_conv_out': [4, 8, 16, 32],
'module__first_fc_out': [128, 256, 512],
}
```

```python
"""
Define `GridSearchCV`.
4 lrs * 7 max_epochs * 4 module__first_conv_out * 3 module__first_fc_out
* 2 CVs = 672 fits.
"""
gs = GridSearchCV(
    net, params, refit=False, scoring='accuracy', verbose=1, cv=2
)

counter = 0
# Run each fit for 2 batches. So, if we have `n` fits, then it will
# actually for `n*2` times. We have 672 fits, so total,
# 672 * 2 = 1344 runs.
search_batches = 2
"""
This will run `n` (`n` is calculated from `params`) number of fits
on each batch of data, so be careful.
If you want to run the `n` number of fits just once,
that is, on one batch of data,
add `break` after this line:
    `outputs = gs.fit(image, labels)`

Note: This will take a lot of time to run
"""
```

```python
for i, data in enumerate(train_loader):
    counter += 1
    image, labels = data
    image = image.to(device)
    labels = labels.to(device)
    outputs = gs.fit(image, labels)
    # GridSearch for `search_batches` number of times.
    if counter == search_batches:
        break
```

Ejercicio de programación

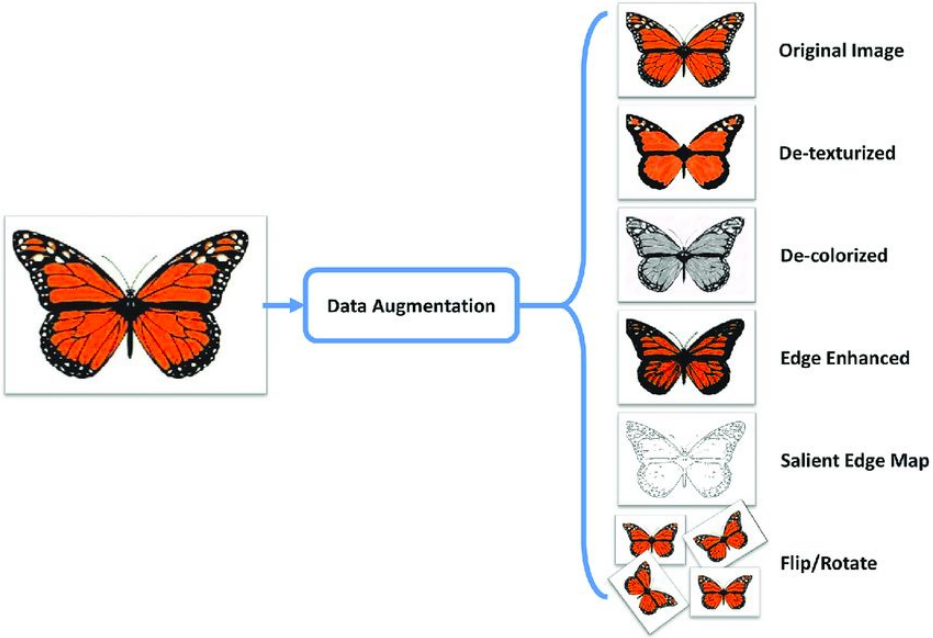
AutoAugment. Motivación

En la práctica habitual se suele usar data augmentation sobre los conjuntos de imágenes, buscando así mejorar el comportamiento de los modelos.
Esto permite aumentar la cantidad de pares entrada y salida, a partir de pequeñas modificaciones en las imágenes existentes.
Como resultado también permite evitar el overfitting en el entrenamiento.

AutoAugment: Learning Augmentation Policies from Data [Link]

AutoAugment. Motivación

Sin embargo, la aplicación de las distintas políticas de augmentacion se suele hacer en base a prueba y error, partiendo de hipótesis que supongan una posible mejora en el rendimiento del modelo. Estas hipótesis se prueban iterativamente hasta mejorar las métricas de interés, pero en la práctica se está lejos de encontrar la combinación óptima de modificaciones dado el gran tamaño del espacio de búsqueda.



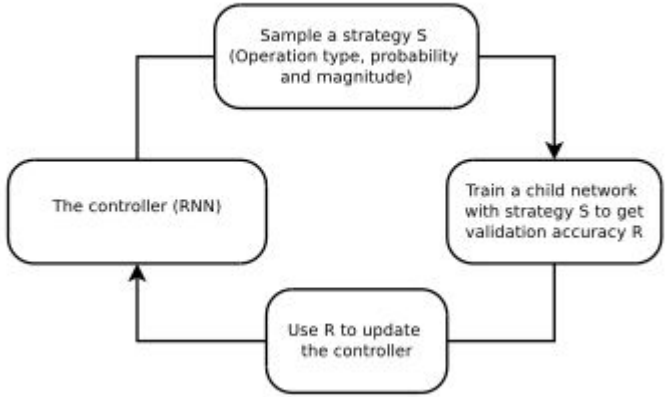
AutoAugment: Learning Augmentation Policies from Data [Link]

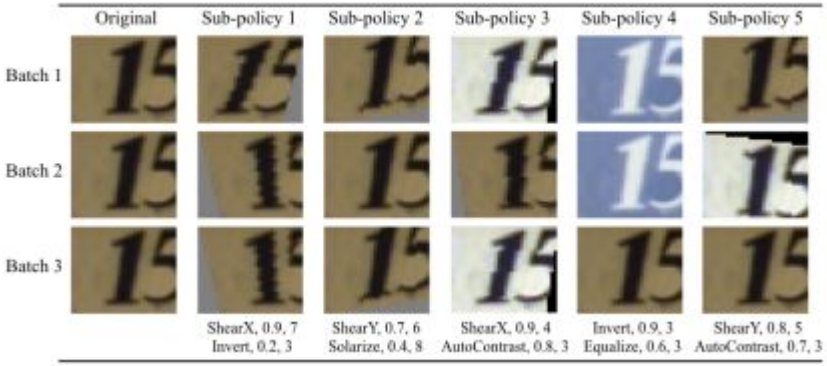
AutoAugment

Investigadores de Google Research propusieron implementar modelo que aprenda las políticas de Data Augmentation que mejor dado un conjunto de datos.

En este caso se mapean cerca de 2,9 x 10^{32} combinaciones posibles para cada una de las sub-políticas de aumentación.

Se usó aprendizaje por refuerzo (PPO) para decidir la mejor política en dicho espacio





AutoAugment: Learning Augmentation Policies from Data Link

AutoAugment. Resultados

Los resultados fueron los esperados, se encontró un conjunto particular de políticas de data augmentation para cada dataset, que resultaron en una mejora en performance en casi todos los datos testeados.

También se probaron mejoras en la performance de datasets custom al hacer transferencia de políticas de data augmentation entrenadas para ImageNet

Dataset	Model	Baseline	Cutout [12]	AutoAugment
CIFAR-10	Wide-ResNet-28-10 [67]	3.9	3.1	2.6±0.1
	Shake-Shake (26 2x32d) [17]	3.6	3.0	2.5±0.1
	Shake-Shake (26 2x96d) [17]	2.9	2.6	2.0±0.1
	Shake-Shake (26 2x112d) [17]	2.8	2.6	1.9±0.1
	AmoebaNet-B (6,128) [48]	3.0	2.1	1.8±0.1
	PyramidNet+ShakeDrop [65]	2.7	2.3	**1.5 ± 0.1**
Reduced CIFAR-10	Wide-ResNet-28-10 [67]	18.8	16.5	14.1±0.3
	Shake-Shake (26 2x96d) [17]	17.1	13.4	**10.0 ± 0.2**
CIFAR-100	Wide-ResNet-28-10 [67]	18.8	18.4	17.1±0.3
	Shake-Shake (26 2x96d) [17]	17.1	16.0	14.3±0.2
	PyramidNet+ShakeDrop [65]	14.0	12.2	**10.7 ± 0.2**
SVHN	Wide-ResNet-28-10 [67]	1.5	1.3	1.1
	Shake-Shake (26 2x96d) [17]	1.4	1.2	**1.0**
Reduced SVHN	Wide-ResNet-28-10 [67]	13.2	32.5	8.2
	Shake-Shake (26 2x96d) [17]	12.3	24.2	**5.9**

Dataset	Train Size	Classes	Baseline	AutoAugment-transfer
Oxford 102 Flowers [43]	2,040	102	6.7	**4.6**
Caltech-101 [15]	3,060	102	19.4	**13.1**
Oxford-IIIT Pets [14]	3,680	37	13.5	**11.0**
FGVC Aircraft [38]	6,667	100	9.1	**7.3**
Stanford Cars [27]	8,144	196	6.4	**5.2**

AutoAugment: Learning Augmentation Policies from Data Link

AutoAugment. Implementacion

```
from autoaugment import ImageNetPolicy
data = ImageFolder(rootdir, transform=transforms.Compose(
                        [transforms.RandomResizedCrop(224),
                         transforms.RandomHorizontalFlip(), ImageNetPolicy(),
                         transforms.ToTensor(), transforms.Normalize(...)]))
loader = DataLoader(data, ...)
```

Para probar este técnica de transferencia de Data Augmentation puede usarse el paquete **autoaugment** que fue desarrollado por los mismo investigadores y que tienen políticas pre entrenadas en varios datasets listas para ser utilizadas

AutoAugment: Learning Augmentation Policies from Data Link

¡GRACIAS POR SU ATENCIÓN!